

3. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

OUTPUT:

```
kannamina_abhishek@LAPTOP-D9F9IT80:~$ nano p3_process.c
kannamina_abhishek@LAPTOP-D9F9IT80:~$ gcc p3_process.c -o p3_process
kannamina_abhishek@LAPTOP-D9F9IT80:~$ ./p3_process
Parent Process
PID : 728
Child PID : 729
Child Process
PID : 729
PPID : 728
Child terminating...
Child completed.
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);

        printf("Child terminating...\n");
    } else {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        sleep(5);

        wait(NULL);
        printf("Child completed.\n");
    }

    return 0;
}
```